Installation

Scaffold a new Almanac site, or add Almanac to an Astro project you already have.

Shravan Goswami / https://shravangoswami.com

Almanac needs Node 22.12 or newer and Astro 7.

`Almanac is not on npm yet, so the commands on this page do not work today. Until the first release, clone the repository and start from the bundled starter, as shown below. This site is built with that same code.`

Start from the repository, today

```
git clone https://github.com/shravanngoswamii/almanac.git
cd almanac
pnpm install
pnpm --filter starter dev
```

`starter/` is a complete minimal site: an `astro.config.mjs`, a `src/content.config.ts`, a homepage, and a `docs/` directory with two example pages. Copy that directory somewhere else and point its `almanac` dependency at your checkout, and you have your own project.

Start a new site, once it is published

```
npm create almanac@latest my-docs
cd my-docs
npm run dev
```

The scaffolder writes the same tree that `starter/` holds. That is the whole starting point.

Add it to an existing Astro project

Once it is published, install the framework and its peer dependencies. Satori and resvg render the OG image, Pagefind builds the search index:

```
npm install almanac satori @resvg/resvg-js pagefind
```

Register the integration:

```
// astro.config.mjs
import { defineConfig } from "astro/config";
import almanac from "almanac";

export default defineConfig({
  site: "https://example.com",
  integrations: [
    almanac({
      title: "My Project",
      social: { github: "https://github.com/you/your-project" },
    }),
  ],
});
```

Declare the collections Almanac renders. The loaders point at top-level directories, so writers never touch `src/`:

```
// src/content.config.ts
import { defineCollection } from "astro:content";
import { blogLoader, blogSchema, docsLoader, docsSchema } from "almanac/content";
```

```
export const collections = {
  docs: defineCollection({ loader: docsLoader(), schema: docsSchema() }),
  blog: defineCollection({ loader: blogLoader(), schema: blogSchema() }),
};
```

Then write `docs/index.md` with a `title` in its frontmatter, and the route exists.

Run the dev server

```
npm run dev
```

This serves the site at `http://localhost:4321`, under whichever `base` your `astro.config.mjs` sets (see Deployment).

Build for production

```
npm run build
```

Astro compiles the site to static HTML, CSS, and JS in `dist/`, and then Almanac's `astro:build:done` hook runs Pagefind over that output to write the search index into `dist/pagefind/`. You don't chain the two commands yourself.

```
If Pagefind can't run, the build logs a warning and finishes anyway. You get a working site without search rather than a failed deploy.
```

Preview the production build

```
npm run preview
```

This serves `dist/`, including the search index, so you can check the real output before deploying.

```
Search only works against a production build. The dev server has no index, and the search dialog deliberately skips loading Pagefind in dev, so the box will open and return nothing. Use `npm run build && npm run preview` to test search locally.
```